Islamic University of Technology

Title: Simulating a Simple Cache Controller FSM

Name: Sadia Afrin Aurthy

ID: 230041114

Section: 01

Submission Date: 16 - 04 - 2026

1. Introduction

In this project, a simulator has been developed in C++ to model a simple cache controller using a finite-state machine. The main goal was to follow the controller style described in the textbook section 5.9 on using a finite-state machine to control a simple cache and to show, for a sequence of read and write requests, how the internal state changes and how those changes affect the interface signals between the processor, the cache, and the memory.

The simulated cache is a blocking cache. In other words, the processor must wait until the cache has finished the request. The implementation keeps the state machine explicit so that each cycle can be observed in the output trace. This makes the simulator useful not only for running the requests but also for understanding why the controller moves from one state to another.

2. Overall Setup of the Simulator

The simulator is organized as a small repository-ready C++ project. The main source file implements the cache controller FSM, the cache array, the memory model, and a simple parser that reads memory requests from an external text file (instructions.txt). The requests are written in a simplified assembly-like form using lw and sw instructions. The simulator takes that text file as input, runs the requests one by one, and writes a cycle-by-cycle trace to an output file (output.txt).

The CPU is modeled as a request source that issues a valid read or write request from a queue and then waits for the Ready signal from the cache controller. The memory never misses, but it takes a fixed number of cycles to read or write a 128-bit block. Here, the memory side has the same fields as the processor side except that the data fields are widened to a full block.

3.Cache Characteristics Used in the Simulation

The simulator follows the straightforward cache. The key characteristics are listed below.

- Direct-mapped cache
- Write-back using write allocate
- Block size is four words (16 bytes or 128 bits)
- Cache size is 16 KiB, so it holds 1024 blocks
- 32-bit addresses
- A valid bit and dirty bit per block

From these values, the address fields were calculated as cache index = 10 bits, block offset = 4 bits, and tag size = 32 - (10 + 4) = 18 bits.

Tag	Index	Block Offset
18 bits	**10 bits**	**4 bits**

Figure-1: Address fields used by the simulator.

4. Interface Signals

The signals between the processor and the cache are kept very close. On the CPU side, the simulator uses a 1-bit Read or Write signal, a 1-bit Valid signal, a 32-bit address, a 32-bit data field from processor to cache, a 32-bit data field from cache to processor, and a 1-bit Ready signal saying the cache operation is complete.

The interface between the memory and the cache has the same fields except that the data fields are widened to 128 bits. Therefore, on the memory side, the simulator uses a 1-bit Read or Write signal, a 1-bit Valid signal, a 32-bit address, 128-bit data from cache to memory, 128-bit data from memory to cache, and a 1-bit Ready signal saying the memory operation is complete.

5. Project Files

File	Purpose
cache_main.cpp	Implements the simple cache, the memory model, the request parser, and the FSM controller.
instructions.txt	Contains the assembly-like memory requests to be simulated.
output.txt	Stores the cycle-by-cycle output trace produced by the simulator.
README.md	Explains how to build, run, and test the program.

6. Finite-State Machine Design

A finite-state machine consists of a set of states and directions on how to change states. The directions are defined by a next-state function, which maps the current state and the inputs to a new state. When a finite-state machine is used for control, each state also specifies a set of outputs that are asserted when the machine is in that state. The simulator uses that same idea directly: the current state determines the control action for the present cycle, and depending on the conditions, the next state is selected.

The controller in this project has four states: Idle, Compare Tag, Write-Back, and Allocate. The meaning of each state in the implementation is given below-

Idle: This state waits for a valid read or write request from the processor. When a request appears, the FSM moves to Compare Tag.

Compare Tag: This state tests whether the requested read or write is a hit or a miss. If the selected block is valid and the tag matches, then it is a hit. For a load, the selected word is read and the Ready signal is set. For a store, the selected word is updated, the dirty bit is set, and the Ready signal is set. If it is a miss, the controller either goes to Write-Back when the old block is dirty or to Allocate when the old block is clean.

Write-Back: This state writes the 128-bit block to memory using the address formed from the old tag and the cache index. The FSM remains in this state until memory asserts Ready. When the memory write is complete, the controller goes to Allocate.

Allocate: The new block is fetched from memory. The FSM remains in this state until the memory read is complete. Then the new block is placed in the cache, and the FSM goes back to Compare Tag so that the original request can finish on the now-present block.

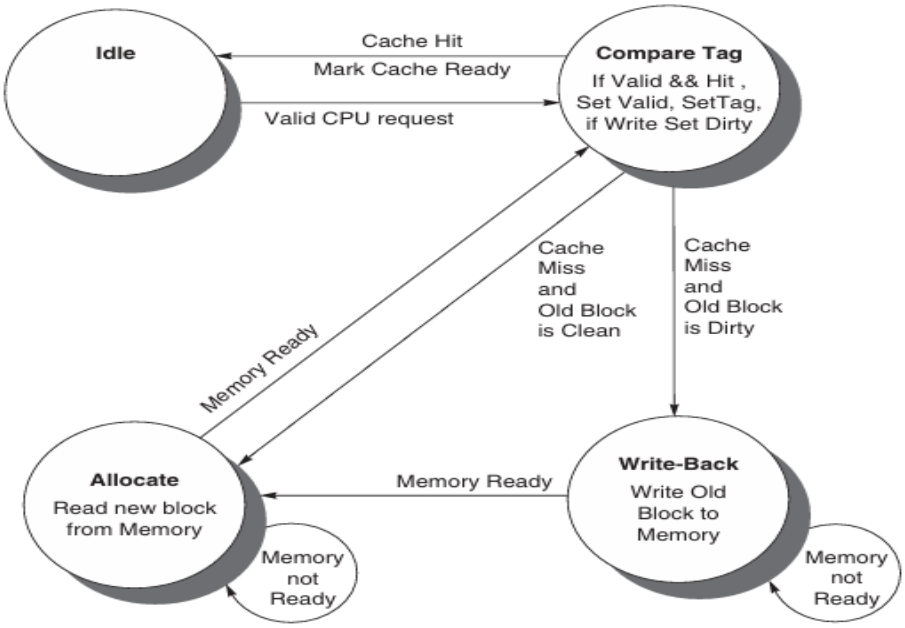

Figure-2: FSM used in the simulator.

7. How the C++ Implementation Maps to the FSM

The code keeps one explicit enum for the current state. Every request is handled cycle by cycle. In each cycle, the simulator first prepares the input signals, then checks whether a memory operation has completed, and finally applies the action of the current state. The output file therefore shows the exact state name, the CPU signals, the memory signals, and the reason for the transition for every cycle.

The cache itself is represented as an array of 1024 cache lines. Each line stores the valid bit, dirty bit, tag, and four 32-bit words. Address decoding is done with helper functions that split the 32-bit address into tag, index, and block offset. A separate memory class stores data per word and can read or write a full four-word block. To keep the results deterministic, unused memory locations are initialized by a simple address-based pattern.

The simulator uses write allocate, so a store miss still fetches the new block first. The store is then completed in the final Compare Tag state after the block has been allocated. This is the common write-allocate behavior for the simple controller.

8. Input Format

The simulator reads requests from an external text file (instructions.txt).

```
lw x1, 0x00000000
lw x2, 0x00000004
sw x3, 0x00000008
lw x4, 0x00004000
lw x5, 0x00000008
sw x6, 0x00008000
lw x7, 0x00004000
```

The chosen sequence's been prepared intentionally so that all important functionalities are exercised: clean miss with allocation, read hit, write hit with dirty-bit update, conflict miss that causes write-back, and another store miss followed by a later dirty replacement.

9. Experimental Scenarios and Output Analysis

Because the cache is direct-mapped, all three base addresses 0x00000000, 0x00004000, and 0x00008000 use the same cache index. They differ only in the tag field. This makes them useful for generating conflict misses deliberately.

Request	Instruction	Main behavior	Cycles used
1	lw x1, 0x00000000	Read miss on an invalid line, then allocate, then hit	5
2	lw x2, 0x00000004	Read hit on the already allocated block	2
3	sw x3, 0x00000008	Write hit, selected word updated, dirty bit set	2
4	lw x4, 0x00004000	Conflict miss, old block dirty, write-back then allocate	7
5	lw x5, 0x00000008	Read miss to block written earlier; value comes back from memory after prior write-back	5
6	sw x6, 0x00008000	Store miss on clean block, allocate first, then store and set dirty	5
7	lw x7, 0x00004000	Conflict miss against dirty block at tag 0x2; write-back then allocate	7

9.1 Scenario 1: First Load to an Empty Cache Line

The first request is lw x1, 0x00000000. Since the line is initially invalid, the controller does not find a hit in Compare Tag. The old block is not dirty, so the FSM goes directly to Allocate. Memory returns the block after two cycles, the line becomes valid, and the controller returns to Compare Tag. On that second visit, the request becomes a hit and the selected word is supplied to the processor with Ready = 1.

```
Cycle   2 | Compare Tag | Cache miss and old block is clean, so the FSM
goes to Allocate.
Cycle   4 | Allocate    | Memory read is complete; FSM returns to Compare
Tag to finish the request.
Cycle   5 | Compare Tag | Cache hit. Data are read from the selected word
and the Ready signal is sent.
```

9.2 Scenario 2: Read Hit and Write Hit on the Same Block

The second request accesses 0x00000004, which is in the same four-word block as the first request. Therefore, it is a straightforward hit. The third request is a store to 0x00000008, which is also in the same block. Since it is a hit, the selected word is updated inside the cache and the dirty bit is set to 1. No memory transaction is required at that time because the cache uses write-back.

9.3 Scenario 3: Dirty Replacement and Write-Back

The request lw x4, 0x00004000 uses the same index as the previous block but a different tag. The line currently holds tag 0x0 and it is dirty because of the earlier store. Therefore, the controller cannot simply discard it. In Compare Tag the miss is detected, the FSM moves to Write-Back, and the 128-bit block is sent to memory. Only after memory asserts Ready does the FSM move to Allocate and fetch the new block. After the new block arrives, Compare Tag is entered again and the load completes.

```
Cycle  11 | Compare Tag | Cache miss and old block is dirty, so the FSM
goes to Write-Back.
Cycle  13 | Write-Back  | Ready signal from memory completes the write; FSM
goes to Allocate.
Cycle  15 | Allocate    | Memory read is complete; FSM returns to Compare
Tag to finish the request.
Cycle  16 | Compare Tag | Cache hit. Data are read from the selected word
and the Ready signal is sent.
```

This scenario clearly shows why the Write-Back state exists. Since the cache uses write-back, the modified word written by sw x3, 0x00000008 remains only in the cache until the dirty block is evicted. The write-back operation sends the whole 128-bit block to memory, not just the single modified word.

9.4 Scenario 4: Verifying that the Write-Back Preserved the Stored Data

After the previous replacement, the request lw x5, 0x00000008 is sent again. At this moment the old block with tag 0x0 is no longer in the cache, so the controller must allocate it again from memory. The important observation is that the value returned for the selected word is now 0x33334333, which is exactly the word written earlier by the store to x3. This demonstrates that the dirty block was correctly written back to memory before replacement.

9.5 Scenario 5: Store Miss with Write Allocate

The request sw x6, 0x00008000 is a store miss. Because the design uses write allocate, the controller first fetches the block from memory in Allocate. Once the block has arrived, the FSM goes back to Compare Tag, the store is performed on the selected word, and the dirty bit is set. This case shows that a store miss does not write directly to memory in this simple controller; instead, the block is allocated and then updated in the cache.

10. Build and Run Procedure

The project can be built and executed with the following commands as well.

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic cache_main.cpp -o cache_fsm
./cache_fsm instructions.txt output.txt
```

The same workflow is also available through the included README.md. The generated output file contains the full cycle-by-cycle trace for all read-write requests in assembly.

11. GitHub Repository

A public GitHub repository is required for this assignment. The project folder prepared for this submission already contains the source file, input file, README, generated trace output file, and report files. The repository link is given below-

`https://github.com/sadiaaurthy/Simple-Cache-Controller-FSM.git`

12. Conclusion

This project successfully simulates a simple cache controller using an explicit finite-state machine. The simulation demonstrates the textbook (section 5.9) behavior of a direct-mapped, write-back, write-allocate cache with the four states Idle, Compare Tag, Write-Back, and Allocate. By using a request file (instructions.txt) and by printing the CPU and memory interface signals cycle by cycle, the program makes it easy to observe how a cache hit completes quickly, how a clean miss requires allocation, and how a dirty miss requires write-back before allocation.

The chosen set of test requests covers all major functionalities of the controller and shows that the internal state changes are consistent with the intended hardware behavior. In particular, the run verifies that a dirty block is preserved through write-back and that later loads can retrieve the updated value from memory. For this reason, the simulator is both a working program and a clear demonstration of the cache-controller FSM described in the course material.